设备 MQTT 接入协议规范（通用部分）

项目	内容
文档版本	V1.0
发布日期	2026-10-23
适用范围	本公司所有支持 MQTT 接入的设备（直连设备与网关设备）
读者对象	负责将本公司设备接入客户自有平台 / 专属云平台的集成开发与运维人员

> 本文档仅描述所有设备共用的 MQTT 接入规则：连接方式、认证、Topic、报文结构和交互流程。各型号设备具体上报哪些数据点、支持哪些命令，请参阅对应设备的《数据点与命令说明》文档。

目录

1. 概述
2. MQTT 协议简介
3. 设备端可配置参数
4. 连接与认证
5. Topic 规范
6. 报文通用约定
7. 遥测数据上报
8. 属性
9. 远程调用（RPC）
10. 可选能力：网关子设备
11. 可选能力：OTA 固件升级
12. 可选能力：自动注册
13. 可选能力：设备认领
14. 设备生命周期与时序
15. 最佳实践
16. 平台侧对接要点
17. 常见问题排查
18. 附录

1. 概述

1.1 设计目标

- 通用：所有设备使用同一套连接、认证与报文框架，各型号只在"数据点"和"命令"层面不同。
- 可配置：服务器地址、认证方式、所有 Topic 均可在设备端配置，便于适配不同客户平台的命名规范。
- 轻量：报文采用 UTF-8 编码的 JSON，结构扁平，便于嵌入式设备生成与解析。
- 可靠：支持断线重连、离线缓存补传、请求/应答关联。

1.2 数据模型

设备与平台之间交换的数据分为三类：

类别	方向	含义	典型示例
遥测（Telemetry）	设备 → 平台	随时间变化的测量值，按时间序列存储	温度、电压、运行状态
属性（Attributes）	双向	相对静态的描述或配置信息，只保留最新值	上行：固件版本、序列号；下行：上报周期、阈值
远程调用（RPC）	双向	请求/应答式的命令交互	重启、开关控制、查询实时状态

属性按来源分为两种：

属性类型	写入方	设备行为	报文中的字段名
设备属性	设备	设备上报，平台存储	`client` / `clientKeys`
下发属性	平台	平台修改后推送给设备，设备也可主动拉取	`shared` / `sharedKeys`

1.3 术语

术语	说明
平台 / 服务端	客户部署的、实现本协议服务端逻辑的 MQTT 接入平台
直连设备	自身直接建立 MQTT 连接的设备
网关	通过一个 MQTT 连接代理多个子设备（如 Modbus、BLE、Zigbee 下挂设备）的设备
子设备	通过网关接入平台的设备，不直接建立 MQTT 连接
访问令牌（Token）	平台为设备分配的唯一身份凭据
`{request_id}`	Topic 中的请求序号占位符，用于请求与应答的配对

2. MQTT 协议简介

MQTT（Message Queuing Telemetry Transport）是 OASIS 标准的轻量级发布/订阅消息协议，运行在 TCP 之上，专为带宽受限、网络不稳定的物联网场景设计。

2.1 基本模型



- 客户端（Client）：本文中即设备或网关，主动连接服务端。
- 服务端（Broker）：负责接收消息并按 Topic 转发。
- 发布（PUBLISH）/ 订阅（SUBSCRIBE）：发送方与接收方通过 Topic 解耦，彼此无需知道对方地址。

2.2 Topic 与通配符

Topic 是以 `/` 分隔的层级字符串，区分大小写，例如 `v1/devices/me/telemetry`。订阅时可使用通配符：

通配符	含义	示例
`+`	匹配一个层级	`v1/devices/me/rpc/request/+` 匹配 `.../request/1`、`.../request/25`
`#`	匹配零个或多个层级，只能出现在末尾	`v1/devices/me/#`

发布消息时 Topic 中不能包含通配符。

2.3 服务质量（QoS）

QoS	语义	说明	本协议建议
0	至多一次	不确认，可能丢失	高频、可容忍丢失的数据
1	至少一次	需 PUBACK 确认，可能重复	缺省值，适用于绝大多数消息
2	恰好一次	四次握手，开销大	不使用；服务端最高授予 QoS 1

2.4 其他关键概念

概念	说明
Keep Alive（心跳）	客户端在该时间间隔内无报文时发送 PINGREQ；服务端在 1.5 倍该时间内未收到任何报文即判定连接断开
Clean Session	为 `true` 时每次连接都是全新会话，重连后需重新订阅
Client ID	客户端标识，同一服务端上必须唯一；相同 Client ID 的新连接会踢掉旧连接
保留消息（Retain）	本协议不使用，设备发布时应置为 `false`
遗嘱消息（Will）	本协议不依赖遗嘱判断在线状态，在线状态由连接本身决定

2.5 协议版本

设备端缺省使用 MQTT 3.1.1，可配置为 3.1 或 5.0。本文所有报文在三个版本下保持一致。

3. 设备端可配置参数

以下参数可通过设备的本地配置界面、配置文件、串口指令或出厂烧录工具进行设置（具体配置方式见各设备说明书）。表中"缺省值"为设备出厂默认值。

3.1 连接参数

参数名	说明	缺省值	取值 / 约束
`mqtt.enable`	是否启用 MQTT 接入	`false`	`true` / `false`
`mqtt.host`	服务端地址（域名或 IP）	空（必须配置）	示例：`mqtt.example.com`
`mqtt.port`	服务端端口	`1883`（启用 TLS 时为 `8883`）	1 ~ 65535
`mqtt.protocol_version`	MQTT 协议版本	`3.1.1`	`3.1` / `3.1.1` / `5.0`
`mqtt.client_id`	客户端标识	设备序列号（SN）	1 ~ 64 字符，全局唯一
`mqtt.keep_alive`	心跳间隔（秒）	`60`	10 ~ 600
`mqtt.clean_session`	是否清除会话	`true`	`true` / `false`
`mqtt.connect_timeout`	建立连接超时（秒）	`10`	3 ~ 60
`mqtt.qos`	上行消息与订阅缺省 QoS	`1`	`0` / `1`
`mqtt.max_payload_size`	单条报文最大字节数	`65536`	不得超过平台限制

3.2 认证参数

参数名	说明	缺省值	取值 / 约束
`auth.type`	认证方式	`access_token`	`access_token` / `basic` / `x509` / `provision`（见第 4、12 章）
`auth.access_token`	访问令牌（`access_token` 模式）	空	由平台分配，1 ~ 128 字符
`auth.username`	用户名（`basic` 模式）	空	—
`auth.password`	密码（`basic` 模式）	空	—

3.3 TLS 参数

参数名	说明	缺省值	取值 / 约束
`tls.enable`	是否启用 TLS	`false`（生产环境强烈建议开启）	`true` / `false`
`tls.version`	最低 TLS 版本	`1.2`	`1.2` / `1.3`
`tls.ca_cert`	用于校验服务端证书的 CA 证书（PEM）	内置公共根证书库	私有 CA 时必须配置
`tls.client_cert`	设备证书（`x509` 模式）	空	PEM 格式
`tls.client_key`	设备私钥（`x509` 模式）	空	PEM 格式，应存放于安全存储区

参数名	说明	缺省值	取值 / 约束
`tls.verify_hostname`	是否校验服务端证书域名	`true`	仅调试时可关闭

3.4 重连与缓存参数

参数名	说明	缺省值	取值 / 约束
`reconnect.min_interval`	首次重连等待（秒）	`1`	≥ 1
`reconnect.max_interval`	最大重连等待（秒）	`60`	≥ `min_interval`
`reconnect.jitter`	重连等待随机抖动比例	`0.2`（±20%）	0 ~ 0.5
`cache.enable`	断线期间是否缓存遥测	`true`	`true` / `false`
`cache.max_records`	最大缓存条数，满后丢弃最旧数据	由设备存储能力决定，见设备说明书	—
`cache.batch_size`	恢复连接后每批补传条数	`50`	1 ~ 500，且单批不超过 `mqtt.max_payload_size`
`rpc.client_timeout`	设备发起 RPC 等待应答超时（秒）	`30`	5 ~ 300
`attr.request_on_connect`	连接成功后是否主动拉取下发属性	`true`	`true` / `false`

3.5 Topic 参数

所有 Topic 均可配置。缺省值见下表，`{request_id}` 等占位符由设备运行时替换。完整说明见第 5 章。

参数名	用途	方向	缺省值
`topic.telemetry`	遥测上报	发布	`v1/devices/me/telemetry`
`topic.attributes_pub`	设备属性上报	发布	`v1/devices/me/attributes`
`topic.attributes_sub`	接收下发属性推送	订阅	`v1/devices/me/attributes`
`topic.attributes_request`	主动拉取属性	发布	`v1/devices/me/attributes/request/{request_id}`
`topic.attributes_response`	属性拉取应答	订阅	`v1/devices/me/attributes/response/+`
`topic.rpc_request_sub`	接收平台下发的 RPC	订阅	`v1/devices/me/rpc/request/+`
`topic.rpc_response_pub`	回复平台 RPC	发布	`v1/devices/me/rpc/response/{request_id}`

参数名	用途	方向	缺省值
`topic.rpc_request_pub`	设备发起 RPC	发布	`v1/devices/me/rpc/request/{request_id}`
`topic.rpc_response_sub`	接收设备发起 RPC 的应答	订阅	`v1/devices/me/rpc/response/+`
`topic.claim`	认领（可选能力）	发布	`v1/devices/me/claim`
`topic.provision_request`	自动注册请求（可选能力）	发布	`/provision/request`
`topic.provision_response`	自动注册应答（可选能力）	订阅	`/provision/response`
`topic.ota_request`	固件分块请求（可选能力）	发布	`v2/fw/request/{request_id}/chunk/{chunk}`
`topic.ota_response`	固件分块数据（可选能力）	订阅	`v2/fw/response/+/chunk/+`
`topic.ota_error`	固件下载错误（可选能力）	发布	`v2/fw/error`
`topic.gw_connect`	子设备上线（网关）	发布	`v1/gateway/connect`
`topic.gw_disconnect`	子设备离线（网关）	发布	`v1/gateway/disconnect`
`topic.gw_telemetry`	子设备遥测（网关）	发布	`v1/gateway/telemetry`
`topic.gw_attributes`	子设备属性上报 / 推送（网关）	双向	`v1/gateway/attributes`
`topic.gw_attributes_request`	子设备属性拉取（网关）	发布	`v1/gateway/attributes/request`
`topic.gw_attributes_response`	子设备属性拉取应答（网关）	订阅	`v1/gateway/attributes/response`
`topic.gw_rpc`	子设备 RPC 下发 / 应答（网关）	双向	`v1/gateway/rpc`
`topic.gw_claim`	子设备认领（网关）	发布	`v1/gateway/claim`

4. 连接与认证

设备支持以下三种常规认证方式，由 `auth.type` 选择。无论哪种方式，每台设备都应使用独立凭据。

4.1 访问令牌（`access_token`，缺省）

CONNECT 字段	取值
Client ID	`mqtt.client_id`
Username	`auth.access_token`
Password	空

最简单的方式，适合快速集成。令牌在网络上以明文传输，生产环境应配合 TLS 使用。

4.2 用户名 / 密码（`basic`）

CONNECT 字段	取值
Client ID	`mqtt.client_id`
Username	`auth.username`
Password	`auth.password`

平台可按 Client ID、用户名或二者组合来识别设备，具体组合规则以客户平台的设备凭据配置为准。

4.3 X.509 双向证书（`x509`）

- 设备通过 TLS 握手提交 `tls.client_cert`，平台根据证书识别设备身份，无需用户名和密码。
- 需同时配置 `tls.enable = true` 与 `tls.client_key`。
- 安全性最高，推荐用于对安全要求高的场景；私钥应存放在安全芯片或加密存储区，不可导出。

4.4 连接结果

CONNACK 返回码（3.1.1）	含义	设备处理
`0`	连接成功	进入订阅流程
`4`	用户名或密码错误	停止快速重试，按最大重连间隔重试并上报本地告警
`5`	未授权（凭据无效或设备被禁用）	同上
其他 / 网络错误	服务端不可用等	按退避策略重连

MQTT 5.0 下对应的 Reason Code 为 `0x00`、`0x86`、`0x87` 等，处理方式相同。

5. Topic 规范

5.1 设计约定

- 缺省 Topic 中的 `me` 表示"当前连接的设备自身"：设备身份由连接凭据确定，Topic 中不需要携带设备标识，因此同一型号的所有设备可使用完全相同的 Topic 配置。
- 请求/应答类 Topic 的最后一级为 `{request_id}`，规则是：谁发起请求，谁向 `request/` 发布；谁做应答，谁向 `response/` 发布。
- 带 `+` 的为订阅用 Topic；发布时由设备替换为具体值。

5.2 直连设备 Topic 一览

功能	设备动作	缺省 Topic	QoS
遥测上报	PUBLISH	`v1/devices/me/telemetry`	1
设备属性上报	PUBLISH	`v1/devices/me/attributes`	1
下发属性推送	SUBSCRIBE	`v1/devices/me/attributes`	1
拉取属性	PUBLISH	`v1/devices/me/attributes/request/{request_id}`	1
拉取属性应答	SUBSCRIBE	`v1/devices/me/attributes/response/+`	1
接收平台 RPC	SUBSCRIBE	`v1/devices/me/rpc/request/+`	1
回复平台 RPC	PUBLISH	`v1/devices/me/rpc/response/{request_id}`	1
设备发起 RPC	PUBLISH	`v1/devices/me/rpc/request/{request_id}`	1
设备 RPC 应答	SUBSCRIBE	`v1/devices/me/rpc/response/+`	1

5.3 自定义 Topic

当客户平台有自己的命名规范时，可修改第 3.5 节中的任意 Topic 参数，例如：

参数	缺省值	自定义示例
`topic.telemetry`	`v1/devices/me/telemetry`	`acme/plant1/{client_id}/telemetry`
`topic.attributes_pub`	`v1/devices/me/attributes`	`acme/plant1/{client_id}/attributes`

自定义时需遵守：

1. 发布用 Topic 不得包含 `+`、`#`；订阅用 Topic 中用于承载 `request_id` 的那一级必须为 `+`。
2. 可使用的占位符：`{client_id}`、`{sn}`（设备序列号）、`{request_id}`、`{chunk}`。
3. 请求类 Topic 与其对应的应答类 Topic 必须同步修改，保证设备能通过最后一级取到 `request_id`。
4. 设备侧修改后，平台侧必须做相同的映射（见第 16 章），否则消息将无法被识别。
5. Topic 总长度不超过 128 字节，建议只使用字母、数字、`-`、`_`、`/`。

6. 报文通用约定

项目	约定
编码	UTF-8 JSON，不带 BOM
根结构	JSON 对象或对象数组（按各章节说明）
键名	区分大小写；建议使用小驼峰或下划线风格，同一产品线保持一致
值类型	字符串、数值（整数 / 浮点）、布尔、JSON 对象；平台按值的 JSON 类型存储
时间戳	字段名 `ts`，Unix 时间戳，毫秒，UTC
大小	单条报文不超过 `mqtt.max_payload_size`（缺省 64 KB）
未知字段	设备与平台都应忽略无法识别的字段，以保证向前兼容

7. 遥测数据上报

Topic：`topic.telemetry`（缺省 `v1/devices/me/telemetry`）

7.1 实时上报（不带时间戳）

平台以收到消息的时间作为数据时间。

```
{
  "temperature": 25.6,
  "humidity": 68,
  "running": true,
  "mode": "auto"
}
```

7.2 带时间戳上报

设备时钟可靠（已完成 NTP 校时）时推荐使用，可消除网络延迟对时间精度的影响。

```
{
  "ts": 1792459200000,
  "values": {
    "temperature": 25.6,
    "humidity": 68
  }
}
```

7.3 批量上报 / 离线补传

断线期间缓存的数据，在恢复连接后以数组形式批量补传，每条必须带原始采集时间 `ts`：

```
[
  { "ts": 1792459200000, "values": { "temperature": 25.6 } },
  { "ts": 1792459260000, "values": { "temperature": 25.8 } },
  { "ts": 1792459320000, "values": { "temperature": 26.1 } }
]
```

场景	是否带 `ts`	说明
实时上报，设备未校时	不带	由平台加盖接收时间
实时上报，设备已校时	建议带	时间更准确
离线补传 / 历史数据	必须带	否则数据时间会错误地变为补传时刻

补传时应按时间升序、分批发送（`cache.batch_size`），并在收到 PUBACK 后再删除本地缓存。

8. 属性

8.1 上报设备属性

Topic：`topic.attributes_pub`（缺省 `v1/devices/me/attributes`）

设备在每次连接成功后、以及属性值变化时上报：

```
{
  "serial_number": "SN20260001",
  "model": "XX-100",
  "firmware_version": "1.2.3",
  "mac": "00:11:22:33:44:55"
}
```

各型号设备的公共属性清单见附录 18.3，型号相关属性见设备专用文档。

8.2 接收下发属性推送

订阅：`topic.attributes_sub`（缺省 `v1/devices/me/attributes`）

平台修改下发属性后实时推送，报文只包含发生变化的键：

```
{ "report_interval": 30, "temp_high_threshold": 45.0 }
```

平台删除下发属性时，推送中可能带有 `deleted` 数组：

```
{ "deleted": ["temp_high_threshold"] }
```

设备收到被删除的键时，应将该配置项恢复为本地缺省值。

8.3 主动拉取属性

用于设备上线后同步平台侧最新配置（推送只在变化时发生，离线期间的变化需要通过拉取获得）。

订阅：`topic.attributes_response`（缺省 `v1/devices/me/attributes/response/+`） 发布：`topic.attributes_request`（缺省 `v1/devices/me/attributes/request/{request_id}`）

请求：

```
{
  "clientKeys": "firmware_version,serial_number",
  "sharedKeys": "report_interval,temp_high_threshold"
}
```

字段	必填	说明
`clientKeys`	否	逗号分隔的设备属性键
`sharedKeys`	否	逗号分隔的下发属性键

两个字段都省略时返回该设备的全部属性。

应答（Topic 末级与请求的 `request_id` 相同，例如 `v1/devices/me/attributes/response/7`）：

```
{
  "client": { "firmware_version": "1.2.3", "serial_number": "SN20260001" },
  "shared": { "report_interval": 30, "temp_high_threshold": 45.0 }
}
```

平台上不存在的键不会出现在应答中。

> 注意：`topic.attributes_sub` 与 `topic.attributes_response` 是两个不同的下行通道，设备必须同时订阅，才能既收到实时推送又收到拉取应答。

---

9. 远程调用（RPC）

9.1 平台 → 设备（命令下发）

```
设备                                                平台
 |  SUBSCRIBE  v1/devices/me/rpc/request/+           |
 | ◀──────── v1/devices/me/rpc/request/42 ──────── |  {"method":"setMode","params":{"mode":"eco"}}
 |  执行命令                                          |
 | ──────── v1/devices/me/rpc/response/42 ────────▶ |  {"result":"ok"}
```

请求报文：

```
{
  "method": "setMode",
  "params": { "mode": "eco" }
}
```

字段	类型	说明
`method`	字符串	方法名，各型号支持的方法见设备专用文档
`params`	任意 JSON	方法参数，无参数时可为 `{}` 或省略

应答报文：任意合法 JSON，通用约定如下：

```
// 成功
{ "result": "ok", "data": { "mode": "eco" } }

// 失败
{ "error": "UNSUPPORTED_METHOD", "message": "method setFoo is not supported" }
```

要求：

- 设备必须使用与请求相同的 `request_id` 回复。
- 应在平台超时前回复；耗时较长的操作应先回复"已受理"，结果再通过遥测或属性上报。
- 对于不支持的方法，应回复错误而不是不回复。
- 平台也可下发"单向"命令（不需要应答），设备回复也不会产生副作用，因此统一回复即可。

9.2 设备 → 平台（设备发起请求）

用于向平台查询信息，例如获取服务器时间、拉取配置文件地址等。可调用的方法由客户平台定义。

```
设备                                                平台
 |  SUBSCRIBE  v1/devices/me/rpc/response/+          |
 | ──────── v1/devices/me/rpc/request/1 ──────────▶ |  {"method":"getTime","params":{}}
 | ◀──────── v1/devices/me/rpc/response/1 ──────── |  {"time":1792459200000}
```

- `request_id` 由设备生成，建议从 1 开始递增，回绕后重新计数，同一时刻未完成的请求不得重号。
- 超过 `rpc.client_timeout` 未收到应答时，设备视为失败并释放该 `request_id`。平台侧超时时可能返回 `{"error":"timeout"}`。

---

10. 可选能力：网关子设备

> 仅适用于具备网关功能的设备。

网关以自身的凭据建立一个 MQTT 连接，代理所有下挂子设备。子设备通过 payload 中的子设备名称区分，不出现在 Topic 中。子设备名称在客户平台内必须唯一，建议使用"网关 SN + 子设备地址"或子设备自身 SN。

10.1 网关 Topic 一览

功能	网关动作	缺省 Topic
子设备上线	PUBLISH	`v1/gateway/connect`
子设备离线	PUBLISH	`v1/gateway/disconnect`
子设备遥测	PUBLISH	`v1/gateway/telemetry`
子设备属性上报	PUBLISH	`v1/gateway/attributes`
子设备下发属性推送	SUBSCRIBE	`v1/gateway/attributes`
拉取子设备属性	PUBLISH	`v1/gateway/attributes/request`
拉取属性应答	SUBSCRIBE	`v1/gateway/attributes/response`
子设备 RPC 下发 / 应答	SUBSCRIBE / PUBLISH	`v1/gateway/rpc`
子设备认领	PUBLISH	`v1/gateway/claim`

10.2 子设备上线 / 离线

```
// v1/gateway/connect
{ "device": "GW001-mb-01", "type": "temperature-sensor" }

// v1/gateway/disconnect
{ "device": "GW001-mb-01" }
```

字段	必填	说明
`device`	是	子设备名称；首次出现时平台自动创建该子设备
`type`	否	子设备类型，平台可据此对子设备分类、套用对应的数据模型

子设备的在线状态完全由网关上报；网关自身断线时，平台将其所有子设备视为离线。网关只有在发送 `connect` 后才会收到该子设备的下行消息。

10.3 子设备遥测

```
{
  "GW001-mb-01": [
    { "ts": 1792459200000, "values": { "temperature": 25.6, "humidity": 68 } }
  ],
  "GW001-mb-02": [
    { "values": { "temperature": 31.2 } }
  ]
}
```

结构为 `{ "子设备名称": [遥测数组] }`，数组元素格式与第 7 章相同。

10.4 子设备属性上报

```
{
  "GW001-mb-01": { "firmware_version": "1.0.4", "slave_address": 1 },
  "GW001-mb-02": { "firmware_version": "1.0.4", "slave_address": 2 }
}
```

10.5 接收子设备下发属性

```
{ "device": "GW001-mb-01", "data": { "report_interval": 30 } }
```

`data` 的格式与第 8.2 节相同（可能包含 `deleted` 数组），网关负责将其转换为子设备的本地协议下发。

10.6 拉取子设备属性

请求（`v1/gateway/attributes/request`）：

```
// 拉取单个键
{ "id": 1, "device": "GW001-mb-01", "client": false, "key": "report_interval" }

// 拉取多个键
{ "id": 2, "device": "GW001-mb-01", "client": false, "keys": ["report_interval", "temp_high_threshold"] }
```

字段	必填	说明
`id`	是	请求序号，由网关生成，用于匹配应答
`device`	是	子设备名称
`client`	是	`true` 拉取设备属性；`false` 拉取下发属性
`key` / `keys`	二选一	单个键（字符串）或多个键（数组）

应答（`v1/gateway/attributes/response`）：

```
// 对应单个键请求
{ "id": 1, "device": "GW001-mb-01", "value": 30 }

// 对应多个键请求
{ "id": 2, "device": "GW001-mb-01", "values": { "report_interval": 30, "temp_high_threshold": 45.0 } }
```

10.7 子设备 RPC

下发（网关订阅 `v1/gateway/rpc`）：

```
{
  "device": "GW001-mb-01",
  "data": { "id": 15, "method": "setMode", "params": { "mode": "eco" } }
}
```

应答（网关发布到 `v1/gateway/rpc`）：

```
{ "device": "GW001-mb-01", "id": 15, "data": { "result": "ok" } }
```

应答中的 `id` 必须与下发报文 `data.id` 一致。

---

11. 可选能力：OTA 固件升级

> 仅适用于支持 OTA 的设备。

11.1 升级通知

平台为设备指定新固件后，通过下发属性推送固件信息（第 8.2 节通道）：

键	说明
`fw_title`	固件名称
`fw_version`	固件版本
`fw_size`	固件大小（字节）
`fw_checksum`	校验值
`fw_checksum_algorithm`	校验算法，如 `SHA256`、`MD5`、`CRC32`

设备上线后也应通过第 8.3 节主动拉取上述键，以免错过离线期间的升级任务。

11.2 分块下载

```
1. SUBSCRIBE  v2/fw/response/+/chunk/+
2. PUBLISH    v2/fw/request/{request_id}/chunk/0     payload: "4096"
3. RECEIVE    v2/fw/response/{request_id}/chunk/0    payload: 第 0 块二进制数据
4. PUBLISH    v2/fw/request/{request_id}/chunk/1     payload: "4096"
   ……循环，直到收到的数据长度 < 块大小（或累计长度 = fw_size）
5. 校验整包 → 写入 → 重启 → 上报新版本
```

项目	说明
`request_id`	设备自定，本次升级过程内保持不变，新一轮升级时更换
`chunk`	块序号，从 0 开始递增
请求 payload	块大小（字节数）的十进制字符串，建议 1024 ~ 16384，且小于 `mqtt.max_payload_size`；为空时平台一次返回整个文件
响应 payload	原始二进制数据（非 JSON）
下载出错	向 `v2/fw/error` 发布错误描述文本

11.3 状态上报

升级过程中通过遥测上报状态，便于平台展示进度：

```
{ "fw_state": "DOWNLOADING" }
```

`fw_state`	含义
`DOWNLOADING`	开始下载
`DOWNLOADED`	下载完成
`VERIFIED`	校验通过
`UPDATING`	正在写入 / 重启
`UPDATED`	升级成功
`FAILED`	失败，同时上报 `fw_error` 说明原因

升级成功、设备重启后上报当前版本：

```
{ "current_fw_title": "XX-100", "current_fw_version": "1.3.0", "fw_state": "UPDATED" }
```

---

12. 可选能力：自动注册

> 适用于批量部署：设备出厂只烧录"注册密钥"，首次上线时向平台申请自己的独立凭据，免去逐台在平台建档。

12.1 前置条件

客户平台为该批设备开启自动注册，并生成一组注册密钥对，在设备端配置：

参数名	说明	缺省值
`provision.key`	注册 Key，标识设备所属的产品 / 批次	空
`provision.secret`	注册 Secret，用于验证合法性	空
`provision.device_name`	申请注册的设备名称	设备序列号（SN）
`provision.credentials_type`	申请的凭据类型	`ACCESS_TOKEN`

12.2 流程

```
1. CONNECT     Username = "provision"（Password 为空）
2. SUBSCRIBE   /provision/response
3. PUBLISH     /provision/request
4. RECEIVE     /provision/response
5. 保存凭据 → DISCONNECT → 使用新凭据按第 4 章重新连接
```

注册连接只允许进行注册操作，不能上报业务数据。

请求：

```
{
  "deviceName": "SN20260001",
  "provisionDeviceKey": "<provision.key>",
  "provisionDeviceSecret": "<provision.secret>"
}
```

字段	必填	说明
`deviceName`	否	设备名称，缺省时由平台生成
`provisionDeviceKey`	是	注册 Key
`provisionDeviceSecret`	是	注册 Secret
`credentialsType`	否	`ACCESS_TOKEN`（缺省）/ `MQTT_BASIC` / `X509_CERTIFICATE`
`token`	否	`ACCESS_TOKEN` 时由设备自行指定令牌；省略则由平台生成
`clientId`、`username`、`password`	否	`MQTT_BASIC` 时由设备指定
`hash`	否	`X509_CERTIFICATE` 时为设备证书（PEM）

成功应答：

```
{
  "status": "SUCCESS",
  "credentialsType": "ACCESS_TOKEN",
  "credentialsValue": "a1B2c3D4e5F6g7H8"
}
```

`MQTT_BASIC` 时 `credentialsValue` 为对象：`{"clientId":"...","userName":"...","password":"..."}`。

失败应答：

```
{ "status": "NOT_FOUND", "errorMsg": "Provision data was not found!" }
```

status	含义	设备处理
`SUCCESS`	成功	持久化凭据，切换 `auth.type`，重连
`NOT_FOUND`	注册密钥无效或未开启注册	停止注册，提示人工检查配置
`FAILURE`	其他失败（如设备名称已存在）	按最大重连间隔重试，超过次数后停止

12.3 安全要求

- 注册 Secret 应在产线通过烧录工具注入，不得硬编码在固件中。
- 注册成功后的凭据须写入非易失存储；之后不再使用注册连接。

---

13. 可选能力：设备认领

> 适用于"设备先出厂、后由终端用户绑定到自己账户"的场景。

13.1 流程

1. 设备（通常在用户按下设备上的配对键后）发布认领信息，开启一个限时认领窗口。
2. 终端用户在客户平台上输入设备名称和认领密钥，完成绑定。
3. 窗口过期后需重新发起。

13.2 报文

Topic：`topic.claim`（缺省 `v1/devices/me/claim`）

```
{ "secretKey": "8F3K2Q", "durationMs": 600000 }
```

字段	必填	说明
`secretKey`	否	认领密钥，用户在平台输入的值必须与之一致；省略时为空字符串
`durationMs`	否	认领窗口时长（毫秒），省略时使用平台缺省值

网关代理子设备认领（`v1/gateway/claim`）：

```
{
  "GW001-mb-01": { "secretKey": "8F3K2Q", "durationMs": 600000 },
  "GW001-mb-02": { "secretKey": "7HJ1PZ", "durationMs": 600000 }
}
```

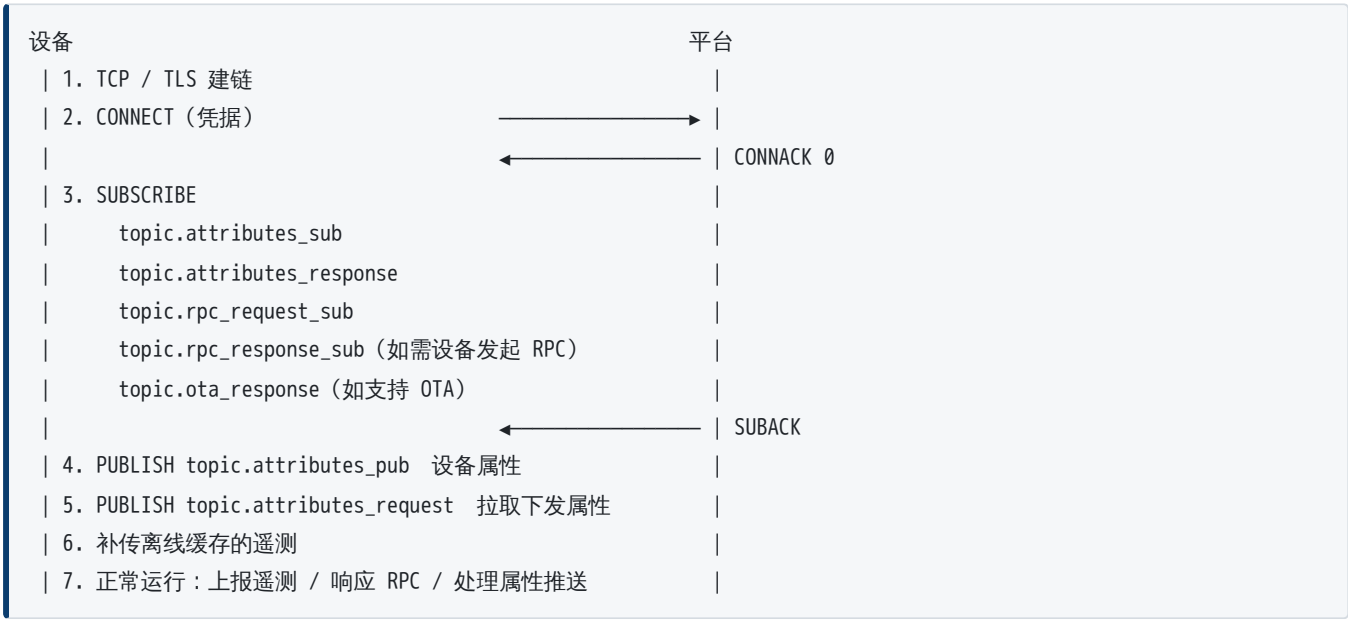
13.3 认领密钥的来源（任选）

方式	说明
标签印刷	出厂时随机生成，印在设备铭牌或二维码上
现场生成	用户按键后设备随机生成并在屏幕 / App 上显示
硬件派生	由 SN、MAC 等经密钥派生得到

不建议使用空密钥，除非设备部署在完全可信的内网。

14. 设备生命周期与时序

14.1 上线流程



必须先订阅、再上报或拉取，否则可能错过应答。

14.2 重连策略

断线后采用带随机抖动的指数退避：

```
wait = min(reconnect.max_interval, reconnect.min_interval × 2^n) × (1 ± reconnect.jitter)
```

例：缺省参数下依次约等待 1、2、4、8、16、32、60、60……秒。连接成功后 n 归零。随机抖动可避免大量设备在平台恢复后同时重连。

14.3 在线状态

平台以 MQTT 连接状态判断设备在线；设备离线的最长感知时间约为 1.5 × `mqtt.keep_alive`。

15. 最佳实践

15.1 安全

- 生产环境一律启用 TLS 1.2 及以上，并校验服务端证书和域名。
- 一机一密：每台设备独立凭据，禁止多台设备共用令牌或证书。
- 凭据、私钥存放在安全存储区，日志中不得打印完整凭据。
- 平台侧应限制每台设备只能访问自身的 Topic。

15.2 可靠性

- 业务消息使用 QoS 1；平台侧按 `ts` + 键对重复数据去重，设备侧对重复收到的 RPC（相同 `request_id`）不应重复执行有副作用的操作。
- 开启离线缓存，补传必须携带原始时间戳。
- 设备应通过 NTP 校时；未校时时不要上报 `ts`。
- RPC 处理不得阻塞 MQTT 收发线程，耗时操作异步执行。

15.3 性能与流量

- 合理设置上报周期，优先"变化上报 + 周期兜底"，避免无变化的高频上报。
- 多个数据点合并在一条消息中上报，而不是一个数据点一条消息。
- 静态信息使用属性，只在变化时上报；测量值使用遥测。
- 单条报文控制在数 KB 以内；补传数据分批发送。
- 平台可能对单设备的消息速率做限制，超限消息会被丢弃或连接被断开。

15.4 兼容性

- 忽略无法识别的字段和不支持的 RPC 方法（回复错误）。
- 新增数据点时只新增键，不修改已有键的含义和类型。
- Client ID 保持稳定，不要每次连接随机生成，以便排查问题。

---

16. 平台侧对接要点

本节供客户平台的集成人员参考，说明平台需要实现或配置的内容：

项目	要求
设备身份	按第 4 章的认证方式，从 CONNECT 或 TLS 证书中识别设备，Topic 中的 `me` 视为当前设备
Topic 映射	若修改了设备端 Topic 参数，平台需将新 Topic 映射到相应功能（遥测、属性、RPC 等）
订阅授权	允许设备订阅第 5.2 节（及网关、OTA 相关）的下行 Topic
下行推送	下发属性变化时按第 8.2 节推送；RPC 按第 9.1 节下发并等待应答
报文限制	单条报文上限不小于设备端 `mqtt.max_payload_size`
QoS	至少支持 QoS 0 和 1

17. 常见问题排查

现象	可能原因	排查方法
CONNACK 返回 4 / 5	凭据错误、设备被禁用	核对令牌是否有多余空格或换行；确认平台上设备状态
连接后立即断开	Client ID 冲突、订阅了无权限的 Topic	确认 Client ID 唯一；检查订阅列表
TLS 握手失败	CA 证书不匹配、设备时间错误导致证书"未生效"	检查 `tls.ca_cert`；确认设备时间
数据上报成功但平台无数据	Topic 拼写错误、JSON 格式错误、平台未保存数据	用 MQTT 调试工具订阅同一 Topic 对比；校验 JSON
数据时间不对	补传数据未带 `ts`，或 `ts` 用了秒而非毫秒	检查 `ts` 为 13 位毫秒时间戳
收不到 RPC	未订阅 `topic.rpc_request_sub`，或订阅在 CONNECT 前发送	确认上线流程顺序
属性拉取无应答	未订阅 `topic.attributes_response`，或 Topic 被自定义但未同步修改	请求和应答 Topic 同步修改
收不到属性推送	只订阅了应答通道	同时订阅 `topic.attributes_sub`
OTA 卡住	未订阅 `topic.ota_response`，或块大小超过报文上限	减小块大小
子设备收不到下行	网关未先发送 `connect`	子设备上线后先发 `connect`

18. 附录

18.1 示例代码（Python，paho-mqtt 1.x）

以下示例演示参数化配置、上线流程、遥测上报、属性处理和 RPC 应答，可作为联调参考。

```
import json, random, time
import paho.mqtt.client as mqtt

CFG = {
    "mqtt.host": "mqtt.example.com",
    "mqtt.port": 1883,
    "mqtt.client_id": "SN20260001",
    "mqtt.keep_alive": 60,
    "auth.access_token": "YOUR_ACCESS_TOKEN",
    "topic.telemetry": "v1/devices/me/telemetry",
    "topic.attributes_pub": "v1/devices/me/attributes",
    "topic.attributes_sub": "v1/devices/me/attributes",
    "topic.attributes_request": "v1/devices/me/attributes/request/{request_id}",
    "topic.attributes_response": "v1/devices/me/attributes/response/+",
    "topic.rpc_request_sub": "v1/devices/me/rpc/request/+",
    "topic.rpc_response_pub": "v1/devices/me/rpc/response/{request_id}",
}

def prefix(sub_topic):            # "a/b/+" -> "a/b/"
    return sub_topic.rstrip("+")

state = {"req_id": 0, "report_interval": 10}

def on_connect(client, userdata, flags, rc):
    if rc != 0:
        print("connect failed, rc =", rc)
        return
    for t in ("topic.attributes_sub", "topic.attributes_response", "topic.rpc_request_sub"):
        client.subscribe(CFG[t], qos=1)
    client.publish(CFG["topic.attributes_pub"],
                   json.dumps({"model": "XX-100", "firmware_version": "1.2.3"}), qos=1)
    state["req_id"] += 1
    client.publish(CFG["topic.attributes_request"].format(request_id=state["req_id"]),
                   json.dumps({"sharedKeys": "report_interval"}), qos=1)

def apply_shared(attrs):
    if "report_interval" in attrs:
        state["report_interval"] = int(attrs["report_interval"])
    if "report_interval" in attrs.get("deleted", []):
        state["report_interval"] = 10

def on_message(client, userdata, msg):
    payload = json.loads(msg.payload.decode("utf-8") or "{}")
    if msg.topic == CFG["topic.attributes_sub"]:
        apply_shared(payload)
    elif msg.topic.startswith(prefix(CFG["topic.attributes_response"])):
        apply_shared(payload.get("shared", {}))
    elif msg.topic.startswith(prefix(CFG["topic.rpc_request_sub"])):
        req_id = msg.topic.rsplit("/", 1)[-1]
        if payload.get("method") == "getStatus":
```

```
            resp = {"result": "ok", "data": {"running": True}}
        else:
            resp = {"error": "UNSUPPORTED_METHOD"}
        client.publish(CFG["topic.rpc_response_pub"].format(request_id=req_id),
                       json.dumps(resp), qos=1)

client = mqtt.Client(client_id=CFG["mqtt.client_id"], clean_session=True)
client.username_pw_set(CFG["auth.access_token"])
client.reconnect_delay_set(min_delay=1, max_delay=60)
client.on_connect = on_connect
client.on_message = on_message
client.connect(CFG["mqtt.host"], CFG["mqtt.port"], CFG["mqtt.keep_alive"])
client.loop_start()

while True:
    data = {"temperature": round(25 + random.uniform(-2, 2), 1)}
    client.publish(CFG["topic.telemetry"], json.dumps(data), qos=1)
    time.sleep(state["report_interval"])
```

18.2 联调工具

可使用任意通用 MQTT 客户端工具（如 MQTTX、mosquitto_pub / mosquitto_sub）模拟设备进行联调，例如：

```
# 上报遥测
mosquitto_pub -h mqtt.example.com -p 1883 -i SN20260001 -u YOUR_ACCESS_TOKEN \
  -q 1 -t v1/devices/me/telemetry -m '{"temperature":25.6}'

# 等待 RPC 命令
mosquitto_sub -h mqtt.example.com -p 1883 -i SN20260001-sub -u YOUR_ACCESS_TOKEN \
  -q 1 -t 'v1/devices/me/rpc/request/+' -v
```

18.3 公共设备属性

所有设备上线后至少上报以下设备属性（型号相关的扩展属性见设备专用文档）：

键	类型	说明
`serial_number`	字符串	设备序列号
`model`	字符串	产品型号
`firmware_version`	字符串	当前固件版本
`hardware_version`	字符串	硬件版本
`mac`	字符串	主网卡 MAC 地址（如有）

18.4 RPC 通用错误码

错误码	含义
`UNSUPPORTED_METHOD`	不支持的方法
`INVALID_PARAMS`	参数缺失或格式错误

错误码	含义
`BUSY`	设备忙，稍后重试
`EXECUTION_FAILED`	执行失败，`message` 字段给出原因

18.5 修订记录

版本	日期	说明
V1.0	2026-10-23	首次发布